

Snowflake × Apache Iceberg Tables

Concepts, Architecture, Practical Theory & Interview Preparation

A plain-language guide — not the textbook version — for engineers who need to actually understand and explain Iceberg tables in Snowflake.

1. What Is Apache Iceberg, Really?

Forget the formal definition for a second. Here's the plain version: Apache Iceberg is a way of organizing files on cheap cloud storage (S3, ADLS, GCS) so that they behave like a proper database table — with schema, transactions, and history — even though the actual data is just a pile of Parquet files sitting in a bucket.

Before Iceberg, if you had a huge folder of Parquet files in S3 and wanted to query it like a table, tools like Hive would just look at the folder structure (partition = folder name) and hope for the best. That approach was slow, fragile, and didn't support things like safely updating a single row, or seeing what the table looked like yesterday.

Iceberg fixes that by adding a metadata layer on top of the data files — a set of small JSON/Avro files that track exactly which data files belong to the table, what the schema is, how it's partitioned, and what old versions ('snapshots') exist. That metadata layer is the entire trick. Everything Iceberg is famous for — time travel, safe concurrent writes, schema evolution without rewriting data, fast query planning — comes from that metadata layer, not from the data files themselves.

Why This Matters for Snowflake Users

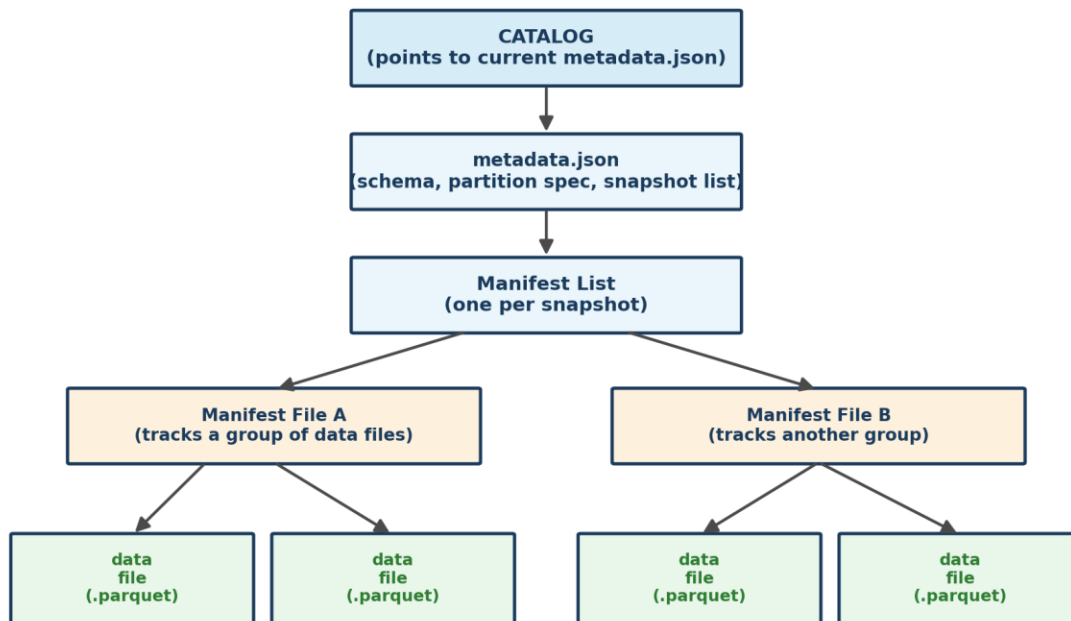
Snowflake's own "native" tables already do all of this internally using their own proprietary format (micro-partitions) that only Snowflake can read. Iceberg tables let you get similar database-like behavior, but using an open, non-proprietary format that other engines — Spark, Trino, Flink, Databricks, Athena — can also read and write. That's the whole pitch: open format, multi-engine access, you own the storage, but you still get warehouse-like features.

In one sentence: Iceberg tables let Snowflake work like a normal Snowflake table on data that physically lives outside Snowflake, in a format multiple tools can share.

2. Iceberg Table Anatomy — The Four Layers

Every Iceberg table is really a stack of pointers, each one pointing down to the next layer. Understanding this stack is the single most useful thing for both real work and interviews, because almost every Iceberg feature (time travel, schema evolution, snapshot isolation) is just "which layer changed."

Apache Iceberg Table Anatomy — Metadata Layers



Query reads the smallest set of manifests/files needed to satisfy the filter — this is how Iceberg "prunes" without scanning every file, and how Time Travel simply points at an older snapshot.

Figure 1: The metadata stack of an Iceberg table — catalog → metadata.json → manifest list → manifest files → data files.

Layer 1 — The Catalog

The catalog's only job is to answer one question: "for table X, what is the current metadata.json file?" That's it. A catalog is essentially a lookup table mapping table names to their latest metadata pointer, plus the ability to atomically swap that pointer (so two people writing at the same time don't corrupt the table). Snowflake, AWS Glue, Hive Metastore, and Polaris are all examples of Iceberg catalogs.

Layer 2 — metadata.json

This file holds the table's schema (current and historical versions), the partition spec, table properties, and — importantly — the full list of snapshots (points-in-time) the table has ever had. Every time you INSERT/UPDATE/DELETE/ALTER the table, a new metadata.json is written and the catalog pointer is atomically swapped to it. The old metadata.json isn't deleted right away — that's what makes time travel possible.

Layer 3 — Manifest List (per snapshot)

Each snapshot has one manifest list — think of it as the table of contents for that version of the table. It lists all the manifest files that make up that snapshot, along with summary statistics (min/max values, row counts) used to skip irrelevant manifests during query planning.

Layer 4 — Manifest Files → Data Files

Each manifest file lists a group of actual Parquet data files, along with per-file statistics. This is what lets Iceberg prune at the file level — if a manifest's stats say a file couldn't possibly contain rows matching your WHERE clause, Iceberg skips reading it entirely, without ever opening the Parquet file.

Interview soundbite: "Iceberg doesn't scan folders — it reads a metadata tree (catalog → metadata.json → manifest list → manifest files) to know exactly which Parquet files to open, which is why it's fast even on huge tables."

3. Iceberg Tables vs. Native Snowflake Tables

This is the question everyone actually cares about: if Snowflake's native tables are already fast, why use Iceberg at all? Here's the honest comparison.

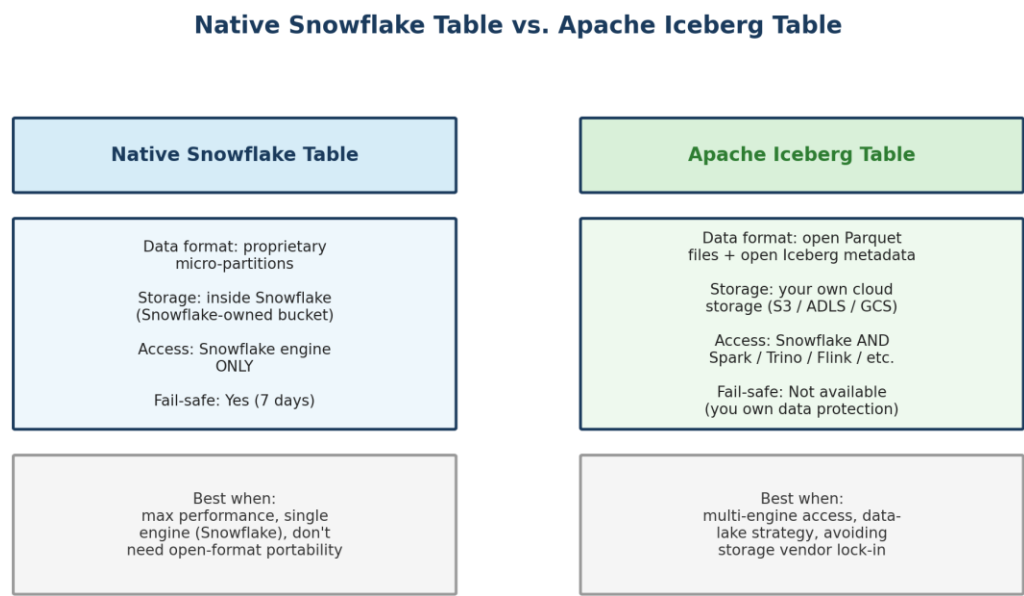


Figure 2: Native Snowflake tables vs. Apache Iceberg tables — the core trade-off is storage ownership and multi-engine access.

Aspect	Native Snowflake Table	Iceberg Table
Storage location	Inside Snowflake (Snowflake-owned)	Your own cloud storage (S3/ADLS/GCS)
File format	Proprietary micro-partitions	Open Apache Parquet
Who can read/write the files	Only Snowflake	Snowflake + Spark, Trino, Flink, etc.
Storage cost billed by Snowflake	Yes	No (except Snowflake-managed storage option)
Fail-safe (7-day recovery)	Yes	Not available — you own recovery
Time Travel	Yes, 0–90 days configurable	Depends on catalog type (see Section 4)
Performance	Slightly faster (native format)	Very close to native for Snowflake-managed
Vendor lock-in	Higher (proprietary format)	Lower (open format, portable)

The honest takeaway: if you will only ever query the data from Snowflake, native tables are simpler and marginally faster. Iceberg earns its complexity when you have other engines (Spark/Databricks/Trino) that also need to read or write the same data, or when you want your data lake to remain usable even if you ever leave Snowflake.

4. The Three Ways Snowflake Works with Iceberg Tables

This is the part that trips up almost everyone, including experienced engineers — including in the reference article this guide draws from. The confusing part is that people assume "who writes the Parquet files" or "where the files sit" determines whether a table is Snowflake-managed or not. It doesn't. The only thing that matters is: who owns the metadata lifecycle — who is allowed to write the next metadata.json and swap the catalog pointer.

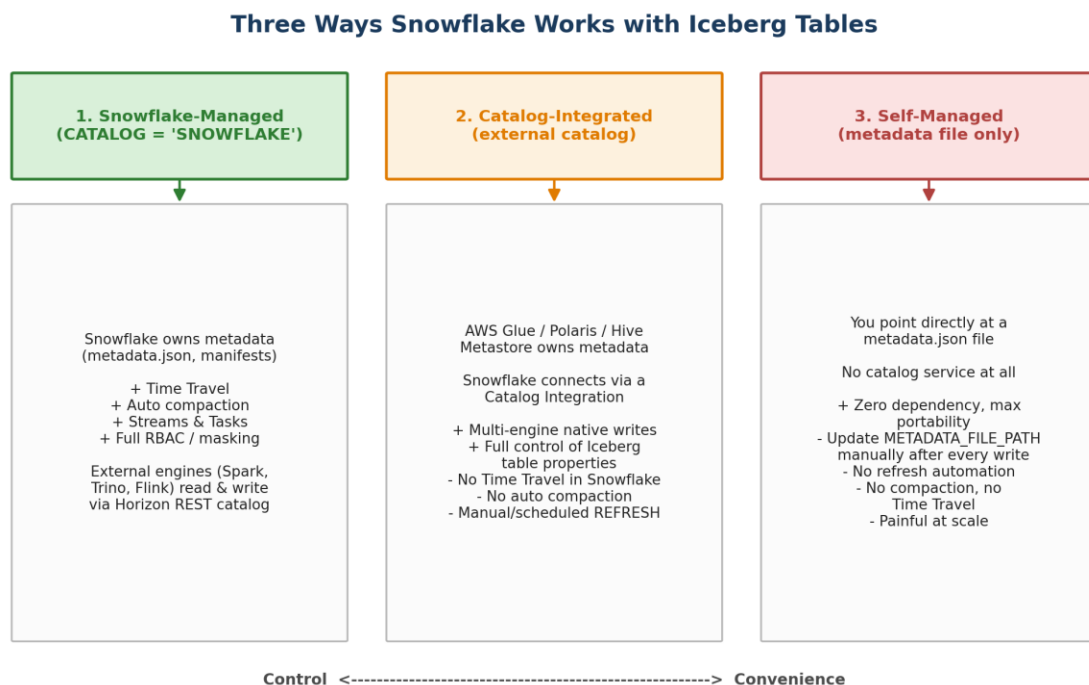


Figure 3: Snowflake supports a spectrum from full automation (Snowflake-managed) to full manual control (self-managed).

4.1 Type 1 — Snowflake-Managed (CATALOG = 'SNOWFLAKE')

Snowflake itself is the Iceberg catalog. You create the table with normal Snowflake DDL, and Snowflake owns and writes all the metadata (metadata.json, manifests, snapshots). External engines like Spark or Trino can still read and write to this table — but they go through Snowflake's own REST catalog endpoint (called Horizon, which is Polaris-compatible) instead of writing metadata directly.

```
CREATE ICEBERG TABLE lakehouse_db.events.app_events (  
  id NUMBER(19,0) NOT NULL,  
  event_type VARCHAR,  
  created_at TIMESTAMP_NTZ(6)
```

```
)
CATALOG = 'SNOWFLAKE'
EXTERNAL_VOLUME = 'my_cloud_volume'
BASE_LOCATION = 'app_events/'
ICEBERG_VERSION = 3;
```

You get:

- Time Travel (controlled by DATA_RETENTION_TIME_IN_DAYS)
- Automatic data compaction (ENABLE_DATA_COMPACTION)
- Automatic manifest compaction & snapshot expiry (always on)
- Streams and Tasks support
- Full Snowflake governance — RBAC, masking policies, row access policies, tags

You give up:

- Can't set raw Iceberg table properties (e.g. write.parquet.compression-codec) — those come from Spark session config instead
- Can't use an external catalog as the source of truth
- External-engine writes have limits: no CTAS from Spark, no base-location override, no equality deletes, no tags/branches, no snapshot rollback, no v2→v3 upgrade from Spark, and no writes if the table has masking/row-access policies or tags

4.2 Type 2 — Catalog-Integrated (CATALOG = '<integration_name>')

An external system — AWS Glue, a standalone Polaris server, or Hive Metastore — owns the metadata.

Snowflake connects to it through a Catalog Integration object, and (as of late 2025) Snowflake can now write to these externally-managed tables too, not just read them.

```
CREATE CATALOG INTEGRATION glue_catalog
  CATALOG_SOURCE = GLUE
  CATALOG_NAMESPACE = 'my_namespace'
  TABLE_FORMAT = ICEBERG
  GLUE_AWS_ROLE_ARN = 'arn:aws:iam::123456789:role/glue-role'
  GLUE_CATALOG_ID = '123456789'
  ENABLED = TRUE;

CREATE ICEBERG TABLE lakehouse_db.events.app_events
  EXTERNAL_VOLUME = 'my_s3_volume'
  CATALOG = 'glue_catalog'
  CATALOG_TABLE_NAME = 'app_events';
```

You get:

- Multiple engines can write natively without going through Snowflake
- Full control over raw Iceberg table properties
- You keep using an existing catalog investment (Glue/Hive/Polaris)
- Consistent snapshot-isolated reads from Snowflake

You give up:

- No Time Travel inside Snowflake
- No automatic compaction — you (or Spark) must run maintenance yourself
- No Snowflake-managed schema evolution unless using a writable configuration
- You must refresh metadata manually (ALTER ICEBERG TABLE ... REFRESH) or configure auto-refresh

4.3 Type 3 — Self-Managed, No Catalog (METADATA_FILE_PATH = '...')

The most manual option. You point Snowflake directly at a specific metadata.json file. There's no running catalog service at all — Snowflake just reads whatever file you told it to.

```
CREATE CATALOG INTEGRATION obj_store_cat
  CATALOG_SOURCE = OBJECT_STORE
  TABLE_FORMAT = ICEBERG
  ENABLED = TRUE;

CREATE ICEBERG TABLE lakehouse_db.events.app_events
  EXTERNAL_VOLUME = 'my_s3_volume'
  CATALOG = 'obj_store_cat'
  METADATA_FILE_PATH = 'app_events/metadata/00003-abc123.metadata.json';
```

You get:

- Zero dependency on any running catalog service — maximum portability
- Works with literally any Iceberg writer

You give up:

- Everything Snowflake normally automates
- You must manually update METADATA_FILE_PATH after every single write, or build your own automation for it
- No automatic refresh, no compaction, no Time Travel
- Operationally painful at any real scale — mostly used for prototyping or one-off/rarely-updated datasets

4.4 Decision Matrix

Situation	Best Choice
Snowflake is your primary/only query engine	Snowflake-Managed
You already have Glue/Hive/Polaris in production	Catalog-Integrated
Multiple engines need native writes without Snowflake in the loop	Catalog-Integrated
You're prototyping or evaluating Iceberg	Self-Managed
Single-writer, rarely-updated dataset	Self-Managed
You want Time Travel + auto-compaction with minimal ops work	Snowflake-Managed
You need maximum portability / zero vendor coupling	Self-Managed

5. How an External Engine Writes to a Snowflake-Managed Table

A common point of confusion: if Snowflake "manages" the table, how can Spark still write to it directly? The answer is that Snowflake exposes a REST API (called Horizon) that speaks the standard Iceberg REST Catalog protocol — the same protocol tools like Polaris use. Spark just treats Snowflake as if it were any other Iceberg REST catalog.

How an External Engine (Spark) Writes to a Snowflake-Managed Iceberg Table

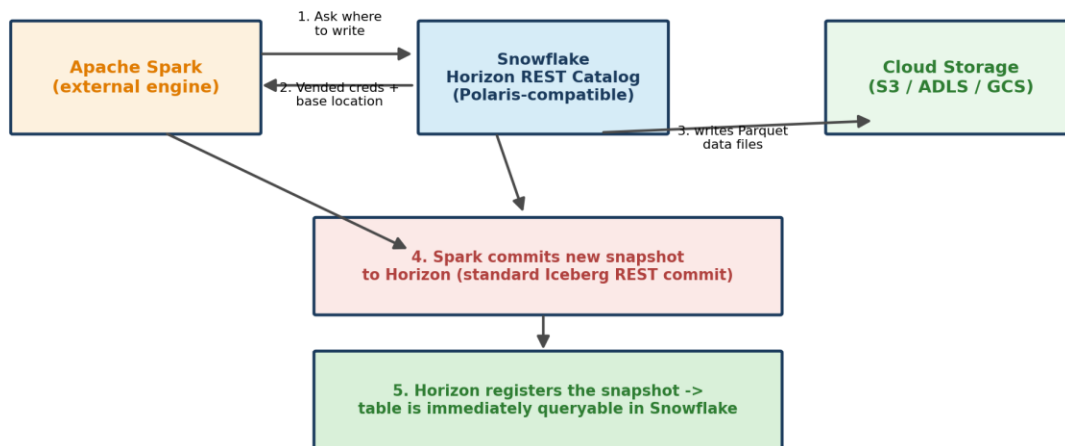


Figure 4: Spark asks Horizon where to write, writes Parquet directly to storage, then commits the new snapshot back through Horizon.

Two details worth remembering because they're easy to get wrong when configuring Spark:

- The Spark catalog's `.warehouse` config value is your Snowflake DATABASE name — not an actual Snowflake virtual warehouse.
- Vended-credential access (letting Spark get temporary storage credentials from Snowflake) requires setting the `X-Iceberg-Access-Delegation` header to `vended-credentials`.

```
spark = (
    SparkSession.builder
        .config("spark.sql.catalog.sf", "org.apache.iceberg.spark.SparkCatalog")
        .config("spark.sql.catalog.sf.type", "rest")
        .config("spark.sql.catalog.sf.uri", CATALOG_URI)
        .config("spark.sql.catalog.sf.warehouse", "LAKEHOUSE_DB") # Snowflake DATABASE, not a
warehouse
        .config("spark.sql.catalog.sf.credential", pat_token)
        .config("spark.sql.catalog.sf.scope", "session:role:DATA_ENGINEER")
        .config("spark.sql.catalog.sf.header.X-Iceberg-Access-Delegation", "vended-credentials")
        .getOrCreate()
)
df.writeTo("sf.EVENTS.APP_EVENTS").using("iceberg").append()
```


Mental model: Snowflake is the catalog brain (decides what the table's official state is), cloud storage is the body (holds the actual bytes), and Spark is just a hand that writes files and then asks the brain to officially recognize the new state.

6. What You Can and Cannot Control (Snowflake-Managed Tables)

6.1 Properties You Set via DDL

Property	When Set	Notes
ENABLE_DATA_COMPACTION	Create or Alter	Turns automatic file compaction on/off
ICEBERG_MERGE_ON_READ_BEHAVIOR	Create or Alter	Controls copy-on-write vs merge-on-read for deletes/updates
TARGET_FILE_SIZE	Create or Alter	AUTO or a fixed byte size for output Parquet files
STORAGE_SERIALIZATION_POLICY	Create ONLY (fixed for life of table)	OPTIMIZED (Snowflake as primary engine) or COMPATIBLE (external-engine reads confirmed needed)
Path layout (flat/hierarchical)	Create	Chooses partition path structure

```
ALTER ICEBERG TABLE app_events SET
  ENABLE_DATA_COMPACTION = TRUE,
  TARGET_FILE_SIZE = 'AUTO';
```

6.2 Properties You Cannot Set from Snowflake DDL

These are standard Iceberg table properties that only exist as Spark session configuration — Snowflake doesn't expose a DDL knob for them:

- write.format.default
- write.parquet.compression-codec
- write.target-file-size-bytes (use Snowflake's own TARGET_FILE_SIZE instead)
- write.metadata.delete-after-commit.enabled
- write.spark.fanout.enabled

6.3 The Identifier Fields (Primary Key) Gap

This is a real, notable limitation worth knowing for interviews: Snowflake-managed Iceberg tables do not expose Iceberg's "identifier fields" (the Iceberg equivalent of a primary key) through SQL DDL. You can set them via Pylceberg, and Spark/Trino/Flink will honor them — but Snowflake's own DESCRIBE TABLE won't show them correctly, and there's no ALTER ICEBERG TABLE ... SET IDENTIFIER FIELDS syntax yet.

Common workarounds: read identifier_field_ids from the Iceberg metadata via Pylceberg at runtime, reference the source system's key configuration, or document the key in the table's COMMENT as a team convention.

7. Key Limitations to Remember

These come up constantly in real projects and in interviews — Iceberg tables are powerful, but they are not a drop-in replacement for native tables in every respect.

Feature	Supported for Iceberg Tables?
Fail-safe	No — not available for any Iceberg table
Hybrid tables	No
Snowflake-native schema evolution (ALTER-based auto evolution)	No — Iceberg has its own schema evolution model
Temporary / Transient table variants	No — Iceberg tables are always "permanent"-like
Cloning	No, for externally managed tables. Possible for Snowflake-managed tables in supported scenarios
Clustering keys	Yes for Snowflake-managed; No for externally managed tables
Streams	Yes (standard + append-only) for Snowflake-managed; only insert-only streams for externally managed
Row-level equality deletes	Not supported (position deletes / deletion vectors used instead)
Multi-statement transactions on externally-managed writes	No — autocommit only

Also worth remembering: Snowflake does not charge Iceberg storage costs when you bring your own external volume — your cloud provider bills you directly for that storage. The exception is when you opt into Snowflake-managed internal storage (`EXTERNAL_VOLUME = 'SNOWFLAKE_MANAGED'`), in which case Snowflake does bill for it.

8. When Should You Actually Use Iceberg Tables?

Skip the marketing angle — here's the practical filter most teams use.

Good fit

- You already have a data lake in S3/ADLS/GCS and multiple compute engines (Spark, Databricks, Trino/Athena) need to query or write the same tables.
- You want to avoid rewriting the same dataset per-engine, or duplicating storage across a warehouse and a lake.
- You care about avoiding long-term storage lock-in to a single vendor's proprietary format.
- You're building a lakehouse architecture where Snowflake is one of several consumers, not the only one.

Weaker fit

- Snowflake is your only and permanent query engine — native tables will be simpler and marginally faster, with Fail-safe included.
- You need heavy multi-statement transactional workloads on externally-managed tables (not supported).
- Your team isn't ready to own external storage lifecycle management (backups, encryption, lifecycle policies) — Snowflake will not do this for you unless you use Snowflake-managed internal storage.

9. Interview Questions & Answers

Simple, spoken-style explanations — the kind you'd actually say out loud in an interview, not a textbook definition. Organized by topic so you can review in blocks.

9.1 Fundamentals

Concept: What Iceberg is

Q: *What is Apache Iceberg, in simple terms?*

A: It's an open table format — a set of metadata files (JSON/Avro) that sit on top of plain Parquet files in cloud storage, making them behave like a real database table: you get schema, transactions (ACID), and history, without needing a proprietary storage engine.

Concept: Why Iceberg exists

Q: *Why not just query a folder of Parquet files directly?*

A: Because a plain folder has no transaction guarantees, no reliable schema history, and query engines have to physically list every file to figure out what exists — which gets painfully slow at scale. Iceberg's metadata layer solves all three problems at once.

Concept: Catalog

Q: *What is an Iceberg catalog, and what does it actually do?*

A: It's the pointer table. For every table, it stores one thing: where is the current metadata.json file? And it can atomically swap that pointer, which is what makes concurrent writes safe. Snowflake, AWS Glue, Hive Metastore, and Polaris can all act as an Iceberg catalog.

Concept: Metadata hierarchy

Q: *Walk me through what happens when Iceberg reads a table.*

A: Engine asks the catalog for the current metadata.json → reads it to get the current snapshot's manifest list → reads the manifest list to find the relevant manifest files (using stored stats to skip irrelevant ones) → reads those manifest files to get the exact list of Parquet data files to open — often skipping most files entirely thanks to stored min/max stats.

Concept: Snapshot

Q: *What is a snapshot in Iceberg?*

A: A complete, point-in-time view of the table — basically a full manifest-list pointer to every data file that existed at that moment. Every write creates a new snapshot. Time Travel is just "go read an older snapshot instead of the latest one."

9.2 Snowflake Integration

Concept: Determining table type

Q: *How do you tell if an Iceberg table is Snowflake-managed or externally managed, just by looking at it?*

A: Run `SHOW ICEBERG TABLES` and check the `catalog_name` and `iceberg_table_type` columns. If `catalog_name = SNOWFLAKE` and `iceberg_table_type = MANAGED`, Snowflake owns the metadata. It does NOT matter where the Parquet files sit or which engine wrote them — that's the trap most people fall into.

```
SHOW ICEBERG TABLES IN SCHEMA LAKEHOUSE_DB.EVENTS;  
-- check: catalog_name, iceberg_table_type, can_write_metadata
```

Concept: Core distinguishing factor

Q: *What actually determines whether a table is 'Snowflake-managed' vs 'self-managed'?*

A: Who owns the metadata lifecycle — i.e., who is allowed to write the next metadata.json and flip the catalog pointer. It is NOT about where the data files live (always external cloud storage) or who writes the Parquet files (can be Spark either way).

Concept: External volume

Q: *What is an external volume in Snowflake, and why is it needed for Iceberg tables?*

A: It's a named Snowflake object that stores the IAM/identity information Snowflake needs to securely read and write to your cloud storage bucket. Since Iceberg data physically lives outside Snowflake, Snowflake needs this to know how to reach it. One external volume can support many Iceberg tables.

Concept: Catalog integration

Q: *What is a catalog integration?*

A: A named Snowflake object that tells Snowflake how to talk to an external Iceberg catalog — for example AWS Glue, or a metadata file directly in object storage. You need one whenever Snowflake is NOT the catalog itself.

Concept: Horizon catalog

Q: *How does Spark write to a table that Snowflake manages?*

A: Snowflake exposes a REST endpoint called Horizon that speaks the standard Iceberg REST Catalog protocol (Polaris-compatible). Spark treats Snowflake exactly like any other Iceberg REST catalog: it asks Horizon where to write, writes Parquet files straight to storage, then commits the new snapshot back to Horizon.

Concept: Time Travel availability

Q: *Does every Iceberg table in Snowflake support Time Travel?*

A: No — only Snowflake-managed Iceberg tables support Snowflake's Time Travel, controlled by DATA_RETENTION_TIME_IN_DAYS. Externally managed tables don't get Snowflake Time Travel; you'd rely on Iceberg's own snapshot history and your external catalog/engine to roll back.

Concept: Fail-safe

Q: *Do Iceberg tables get Fail-safe protection like native Snowflake tables?*

A: No. Fail-safe is not available for any Iceberg table type, Snowflake-managed or not. You are responsible for backup/recovery of the underlying cloud storage yourself (e.g., S3 versioning, lifecycle policies).

Concept: Storage billing

Q: *Does Snowflake charge storage costs for Iceberg tables?*

A: Generally no — if you bring your own external volume (your own S3/ADLS/GCS bucket), your cloud provider bills you directly, not Snowflake. The one exception is Snowflake-managed internal storage (EXTERNAL_VOLUME = 'SNOWFLAKE_MANAGED'), where Snowflake does charge for storage since it's hosting the files itself.

9.3 The Three Table Types

Concept: Snowflake-managed table

Q: *When would you choose a Snowflake-managed Iceberg table over a catalog-integrated one?*

A: When Snowflake is your primary or only query engine and you want the operational burden minimized — automatic compaction, Time Travel, and full RBAC/masking/tags out of the box. You give up fine-grained control

of raw Iceberg write properties and some external-engine write capabilities (no CTAS from Spark, no branch/tag creation, etc.).

Concept: Catalog-integrated table

Q: *When would you choose a catalog-integrated Iceberg table?*

A: When you already have an existing catalog investment (Glue, Hive Metastore, standalone Polaris) or multiple engines need to write natively without Snowflake in the middle. You lose Snowflake Time Travel and automatic compaction — you (or Spark maintenance jobs) own that now.

Concept: Self-managed table

Q: *When would you use the self-managed, no-catalog option (METADATA_FILE_PATH)?*

A: Mostly for prototyping, evaluation, or a single-writer dataset that rarely changes. It has zero dependency on any catalog service, which is great for portability, but you must manually update the METADATA_FILE_PATH after every write — it does not scale operationally.

Concept: Compaction ownership

Q: *Who handles compaction for each of the three table types?*

A: Snowflake-managed: automatic, built-in (can be disabled but is otherwise handled for you). Catalog-integrated: you run it yourself, typically through Spark maintenance procedures. Self-managed: entirely on you — no automation at all.

Concept: Multi-engine writes

Q: *Which table type gives the most native, unrestricted multi-engine write access?*

A: Catalog-integrated. Because an external system like Glue owns the metadata, any Iceberg-compatible engine can write without going through Snowflake's Horizon endpoint or hitting Snowflake's external-write restrictions (e.g., no CTAS, no equality deletes, no branch/tag creation) that apply to Snowflake-managed tables.

9.4 Operational / Practical

Concept: Schema evolution

Q: *Does Snowflake's normal schema evolution feature (auto schema detection) work on Iceberg tables?*

A: No — Snowflake's native automatic schema evolution feature (used for COPY INTO / Snowpipe on regular tables) is not supported for Iceberg tables. Iceberg has its own schema evolution model at the Iceberg metadata level instead (ALTER ICEBERG TABLE ADD COLUMN, etc.).

Concept: Row-level deletes

Q: *How does Snowflake handle row-level deletes/updates on Iceberg tables?*

A: Using Iceberg's standard mechanisms: position deletes for v2 tables, and deletion vectors for v3 tables. Row-level equality deletes are not supported. For best read performance after a lot of deletes, regular compaction/maintenance is needed to clean up old delete files.

Concept: Refreshing externally managed tables

Q: *If an external engine writes new data to a catalog-integrated table, does Snowflake see it immediately?*

A: Not necessarily — you may need to manually run ALTER ICEBERG TABLE ... REFRESH, or configure automated refresh (REFRESH_INTERVAL_SECONDS on the catalog integration) so Snowflake periodically re-reads the latest metadata pointer.

Concept: Cloning

Q: *Can you clone an Iceberg table the same way you clone a native Snowflake table?*

A: Only for certain Snowflake-managed scenarios — cloning is not supported for externally managed Iceberg tables at all.

Concept: Streams on Iceberg

Q: *Can you create a Snowflake stream (for CDC) on an Iceberg table?*

A: Yes for Snowflake-managed tables (standard and append-only streams both work). For externally managed tables, only insert-only streams are supported.

Concept: Clustering keys

Q: *Can you set a clustering key on an Iceberg table?*

A: Yes, but only for tables that use Snowflake as the catalog. Clustering is not supported at all for externally managed Iceberg tables.

Concept: Primary keys / identifier fields

Q: *Does Snowflake support declaring a primary key on an Iceberg table the same way as a native table?*

A: Not really — this is a known gap. Iceberg's own 'identifier fields' concept (its primary-key equivalent) can be set via PyIceberg and honored by Spark/Trino/Flink, but Snowflake's DDL doesn't expose a way to set or view them properly; DESCRIBE TABLE won't show them correctly. Teams typically work around this by storing the key convention in a table COMMENT or looking it up via PyIceberg at load time.

Concept: Iceberg table properties you can't touch

Q: *Can you set arbitrary raw Iceberg table properties like `write.parquet.compression-codec` from Snowflake SQL?*

A: No — those specific low-level Iceberg write properties are controlled through the writing engine's own configuration (e.g., Spark session config), not Snowflake DDL. Snowflake exposes its own equivalents for some things, like `TARGET_FILE_SIZE` instead of `write.target-file-size-bytes`.

9.5 Scenario-Based Questions

Concept: Scenario: choosing a type

Q: *Your company has an existing AWS Glue catalog used by five different Spark jobs, and now wants Snowflake added as a read/write consumer without disrupting the Spark pipelines. Which Iceberg table type do you pick?*

A: Catalog-Integrated — connect Snowflake to the existing Glue catalog via a Catalog Integration. This preserves the existing catalog investment and lets Spark keep writing natively while Snowflake reads (and now can write) through the same metadata source of truth.

Concept: Scenario: minimizing ops

Q: *A team wants Iceberg tables purely for the open-format/portability benefit, but has a small team and cannot take on extra operational overhead like manual compaction. What should they choose?*

A: Snowflake-Managed — it gets them the open Parquet format and eventual portability while Snowflake handles compaction, snapshot expiry, and Time Travel automatically, minimizing day-to-day operational burden.

Concept: Scenario: primary key contract

Q: *A pipeline uses `MERGE INTO` with a business key to upsert into an Iceberg table, and the team wants that key formally declared on the table the way a native table would declare a `PRIMARY KEY`. What do you tell them?*

A: Explain the identifier-fields gap: Snowflake DDL doesn't expose a way to declare or reliably view Iceberg's identifier fields today. Recommend either setting `identifier_field_ids` via PyIceberg (which other engines will

honor) or documenting/enforcing the key as a team convention (e.g., in the table COMMENT and in the MERGE logic itself), since Snowflake's own DESCRIBE TABLE can't be trusted for this.

Concept: Scenario: recovering deleted data

Q: *Someone accidentally deleted a large batch of rows from a Snowflake-managed Iceberg table an hour ago. Can you recover it, and how?*

A: Yes, if it's within the table's Time Travel retention window — use Iceberg/Snowflake Time Travel to query or restore the prior snapshot, since Snowflake-managed tables do support Time Travel via DATA_RETENTION_TIME_IN_DAYS. Note there is no Fail-safe fallback beyond that window for any Iceberg table, unlike native tables.

Concept: Scenario: cross-cloud cost

Q: *Your Snowflake account is in AWS us-east-1, but the Iceberg table's external volume points to a bucket in a different cloud/region. What should you flag to the team?*

A: Cross-cloud/cross-region access is supported, but it isn't free — Snowflake bills cross-region/cross-cloud data transfer under the DATA_LAKE category, and if an external engine reads across regions too, your cloud storage provider will separately bill egress. Flag the added cost before adopting this layout at scale.

9.6 Rapid True/False Round

Statement	True / False
Iceberg table data files can always be read directly from S3 by Spark, even for Snowflake-managed tables.	True — the file format (Parquet) is always open, regardless of who manages the metadata.
Snowflake charges storage costs for every Iceberg table by default.	False — only when using Snowflake-managed internal storage (EXTERNAL_VOLUME = 'SNOWFLAKE_MANAGED').
Fail-safe applies to Snowflake-managed Iceberg tables the same as native tables.	False — Fail-safe is not available for any Iceberg table type.
You can set a clustering key on an externally managed Iceberg table.	False — clustering only works for Snowflake-managed (Snowflake-as-catalog) tables.
A table's type (managed vs. self-managed) is determined by who wrote the Parquet files.	False — it's determined by who owns/writes the metadata, not the data files.
Snowflake's automatic schema evolution feature (for COPY INTO) works on Iceberg tables.	False — not supported; Iceberg uses its own schema evolution mechanism.
Iceberg tables support ACID transactions.	True — this is one of Iceberg's core guarantees, via atomic metadata pointer swaps.
You can declare and see a working PRIMARY KEY on a Snowflake-managed Iceberg table through normal SQL DDL.	False — this is the identifier-fields gap; Snowflake DDL doesn't properly expose this yet.

10. Quick-Reference Syntax Cheat Sheet

Show / Describe

```
SHOW ICEBERG TABLES IN SCHEMA my_db.my_schema;  
DESCRIBE ICEBERG TABLE my_db.my_schema.my_table;
```

Create — Snowflake as catalog

```
CREATE ICEBERG TABLE my_table (  
  id NUMBER, name VARCHAR  
)  
  CATALOG = 'SNOWFLAKE'  
  EXTERNAL_VOLUME = 'my_ext_vol'  
  BASE_LOCATION = 'my_table/'  
  ICEBERG_VERSION = 3;
```

Create — External catalog (e.g. Glue)

```
CREATE CATALOG INTEGRATION glue_cat  
  CATALOG_SOURCE = GLUE  
  CATALOG_NAMESPACE = 'my_ns'  
  TABLE_FORMAT = ICEBERG  
  GLUE_AWS_ROLE_ARN = 'arn:aws:iam::123:role/glue-role'  
  GLUE_CATALOG_ID = '123'  
  ENABLED = TRUE;  
  
CREATE ICEBERG TABLE my_table  
  EXTERNAL_VOLUME = 'my_ext_vol'  
  CATALOG = 'glue_cat'  
  CATALOG_TABLE_NAME = 'my_table';
```

Create — Self-managed (metadata file only)

```
CREATE ICEBERG TABLE my_table  
  EXTERNAL_VOLUME = 'my_ext_vol'  
  CATALOG = 'obj_store_cat'  
  METADATA_FILE_PATH = 'my_table/metadata/0001.metadata.json';
```

Alter tunable properties

```
ALTER ICEBERG TABLE my_table SET  
  ENABLE_DATA_COMPACTION = TRUE,  
  TARGET_FILE_SIZE = 'AUTO';
```

Refresh externally managed metadata

```
ALTER ICEBERG TABLE my_table REFRESH;
```

11. References & Further Reading

- Snowflake Documentation — Apache Iceberg Tables: docs.snowflake.com/en/user-guide/tables-iceberg
- Snowflake Documentation — CREATE ICEBERG TABLE (Snowflake as catalog / external files)
- Vedprakash, "Snowflake-Managed vs. Self-Managed Iceberg Tables: What Actually Determines the Difference," Snowflake Builders Blog, Medium, Jul 2026
- Apache Iceberg official documentation and Table Spec: iceberg.apache.org

This document was compiled for study/interview-preparation purposes. Always verify exact syntax and current limitations against official Snowflake documentation, since Iceberg support evolves quickly.